Dynamic Programming

- Dynamic Programming is an algorithm design technique for solving multi stage optimization problems.
- In Dynamic programming the solution is obtained by making a sequence of decisions.
- It is used when the sub problems are not independent, e.g. when they share the same sub problems(overlapping sub problems)
- Dynamic programming obtains the solution using ***principle of optimality.***
- Principle of optimality states that in an optimal sequence of decisions each sub sequence must also be optimal.

Dynamic Programming vs divide and conquer

- Similar when it comes to dividing a large problem into sub-problems. But here, each sub-problem is solved only once. we store the result of sub-problems in a table for future reuse(**tabulation**).
- The technique of storing sub-problem solutions and reusing in future is called **memorization.**
- In Divide and Conquer, the sub-problems are independent of each other while in case of Dynamic Programming, the sub-problems are not independent of each other.
- Divide and conquer uses top down approach where as dynamic programming uses bottom up approach in solving problems.
- Dynamic Programming is used for solving optimization problems but Divide and conquer is used for solving non optimization problems.

Dynamic Programming vs Greedy method

- The greedy method computes its solution by making its choices in a serial forward fashion, never looking back or revising previous solutions.
- Dynamic programming considers all possible sub sequences in order to obtain the optimal solution.
- It is guaranteed that dynamic programming will generate optimal solution using principle of optimality but in case of greedy algorithm there is no such guarantee.
- Greedy method is efficient and faster than dynamic programming .

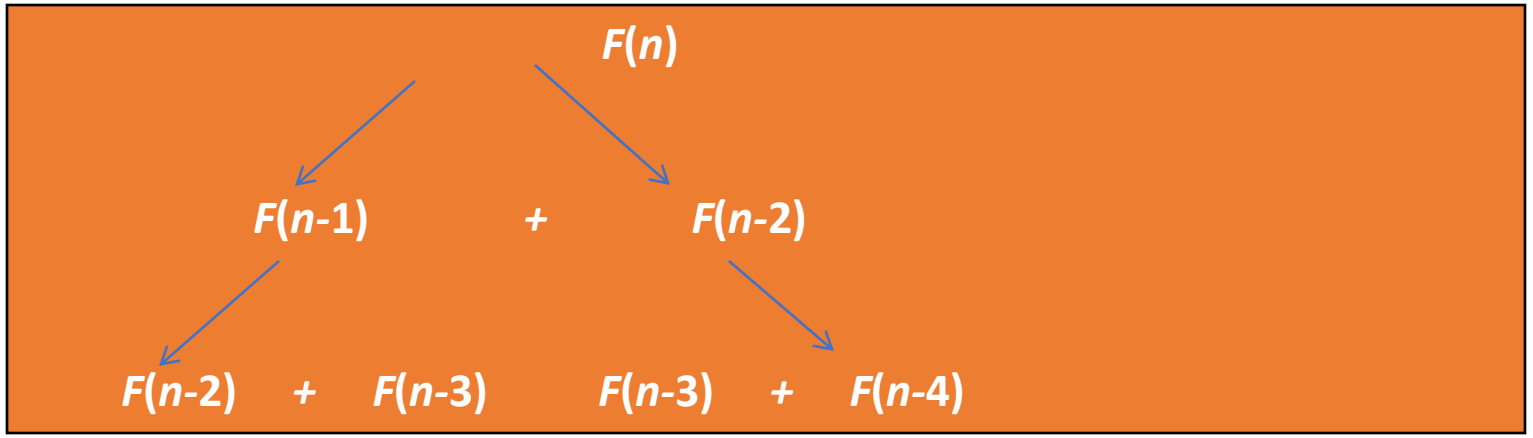
- **Computing the nth Fibonacci number recursively(top down approach)**

  F(n) = F(n-1) + F(n-2)

  F(0) = 0

  F(1) = 1

```
int Fib(int n)
{
   if (n <= 1)
      return 1;
   else
      return Fib(n - 1) + Fib(n - 2);
}
```



The Recurrence relation is

$T(n) = T(n-1) + T(n-2) + 1$

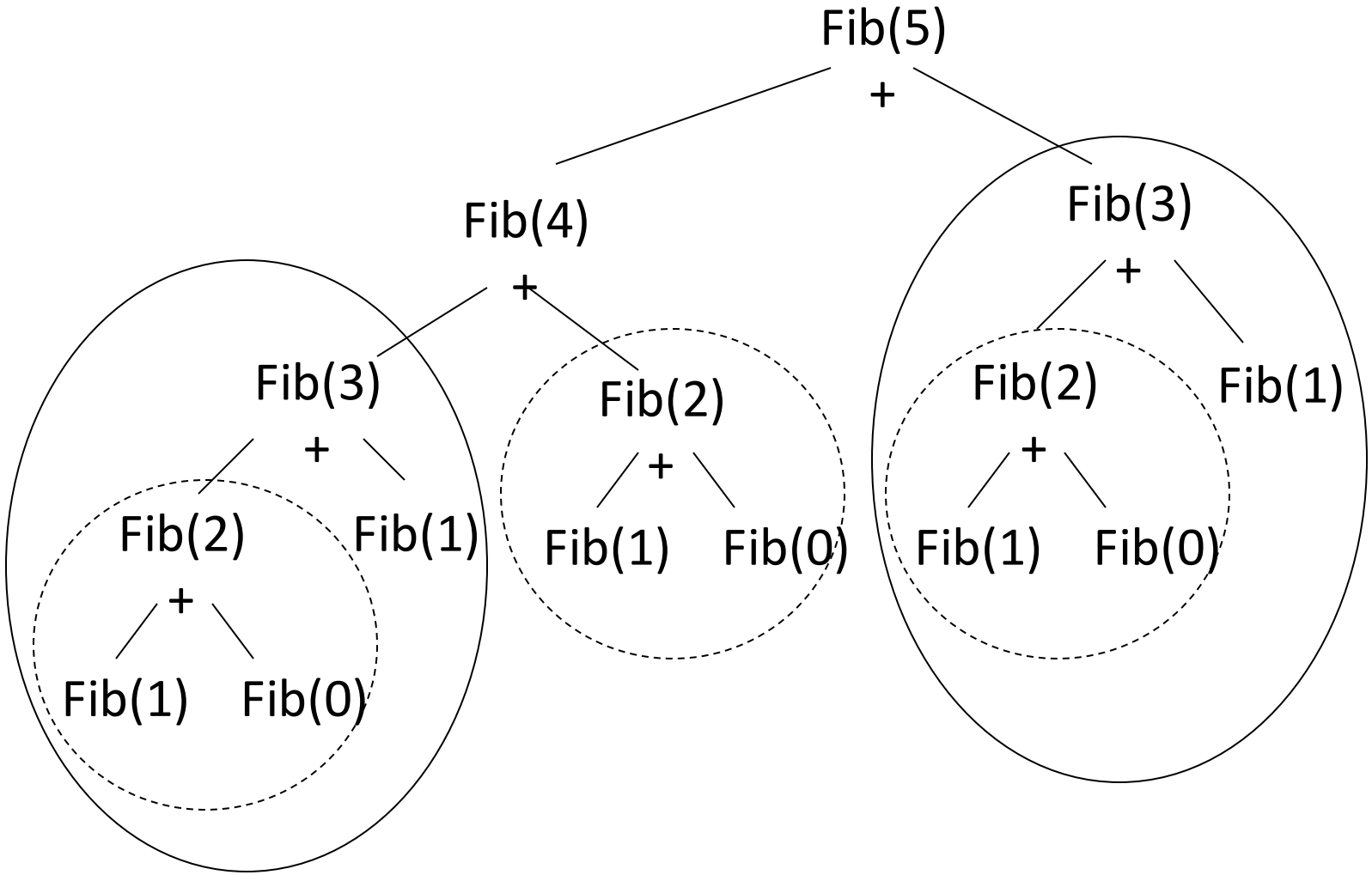
The time complexity is $O(2^n)$.

Computing the nth Fibonacci number using a bottom-up approach:

```
int fib(n)
{
if(n<=1)
retutn n;
else
{
F[0]=0;
F[1]=1;
for (int i=2;i<=n;i++)
{
F[i]=F[i-2]+F[i-1];
}
return(F[n));
}
```

- **Time complexity:T(n)=O(n)**

Overlapping sub problems



Dynamic Programming

- All pairs shortest path problem
- Optimal binary search trees
- 0/1 knapsack problem
- Travelling sales person problem

0/1 knapsack problem

- Given a set of n items, and a knapsack or bag. Each item has a weight wi and profit pi and knapsack has a capacity m .
- In this problem we are not allowed to break items. We either take the whole item or don’t take it.
- If an item i is placed then a profit of pi is earned.
- The objective is determine a subset of items to include in a knapsack so that the total weight is less than or equal to knapsack capacity and the profit value is maximized.

0/1 knapsack problem

- 0/1 knapsack problem is solved using dynamic programming in the following steps:
- 1.Construct the matrix V[i,j] (with (n+1) number of rows and (m+1) number of columns)that stores items and their weights.
- 2.add the items and update the profit values using

  $$V[i,j]= \max\{V[i-1,j],V[i-1,j-wi]+pi\}$$

- 3.Return the maximum profit of adding feasible items to knapsack i.e.V[n,m]

0/1 knapsack problem

- To identify the items that must be put into the knapsack to obtain that maximum profit ,we use the following steps:

1. Consider the last column of the table and start scanning the entries from bottom to top.
2. On encountering an entry whose value is not same as the value stored in the entry immediately above it, mark the row label of that entry.
3. After all the entries are scanned, the marked labels represent the items that must be put into the knapsack

Solve the following 0/1 knapsack problem using dynamic programming:
Knapsack capacity m=5

Item	Weight	Profit
1	2	3
2	3	4
3	4	5
4	5	6

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

T-Table

Optimal solution X=(1,1,0,0)

0/1 knapsack problem

- Solve the following 0/1 knapsack problem using dynamic programming:
- Item weight profit m=8

1	3	2
2	*4*	*3*
3	*6*	*1*
4	*5*	*4*

MAXIMUM PROFIT VALUE:6

Optimal solution X=(1,0,0,1)

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	0	2	2	2	2	2	2
2	0	0	0	2	3	3	3	5	5
3	0	0	0	2	3	3	3	5	5
4	0	0	0	2	3	3	3	5	6






Profit = 6 - 4 = 2

0/1 knapsack problem

- Solve the following 0/1 knapsack problem using dynamic programming:
- Item weight profit m=8

1	2	1
2	*3*	*2*
3	*4*	*5*
4	*5*	*6*

MAXIMUM PROFIT VALUE:8

Optimal solution X=(0,1,0,1)

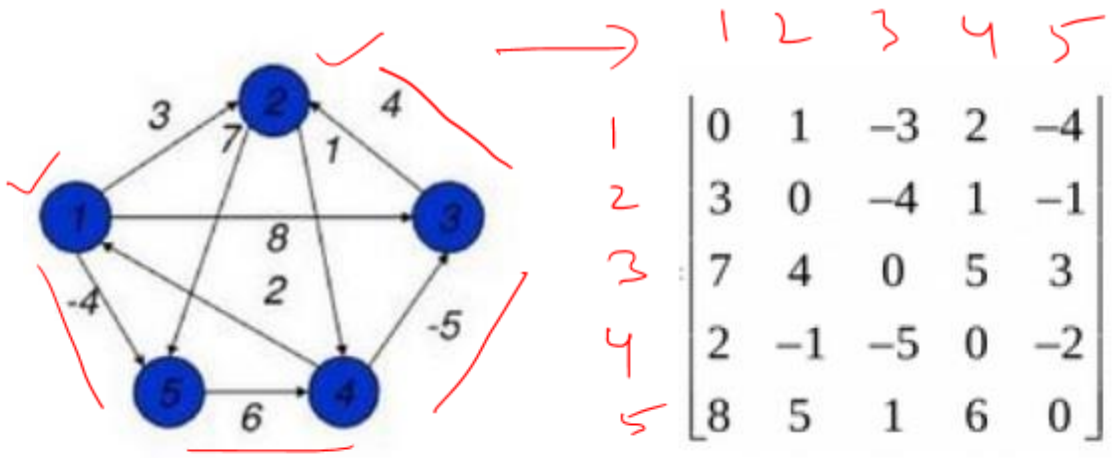
- Apply Dynamic programming method for the following 0/1 knapsack problem n=3,m=6,(p1,p2,p3)=(1,2,4),(w1,w2,w3)=(2,3,3).

```
Algorithm  knapsack(n,m)
{
//p[] ans w[] are the profits and weights of n items and V[i][j]is //the maximum profit value with item i and weight j.
    for (i=0; i <=n; i++)
     {
                for(j=0;j<=m;j++)
                {
                        if(i==0||j==0)
                        {
                                V[i][j]=0;
                        }
                         else if(w[i]<=j) {  V[i][j]= max(V[i-1][j], V[i-1][j-w[i]]+P[i]);
}
                        else {V[i][j]=V[i-1][j];}
                 }
          }
   return(V[n][m];
}
```

Time complexity
T(n)=O(nm)

All pairs shortest path problem

- The all pair shortest path algorithm is also known as **Floyd-Warshall** algorithm is used to solve all pair shortest path problem for a given weighted graph.
- This algorithm will generate a matrix, which will represent the minimum distance from any node to all other nodes in the graph.



	1	2	3	4	5
1	0	1	−3	2	−4
2	3	0	−4	1	−1
3	7	4	0	5	3
4	2	−1	−5	0	−2
5	8	5	1	6	0

All pairs shortest path problem

- At first the output matrix is same as given cost matrix of the graph. After that the output matrix will be updated with all vertices k as the intermediate vertex.
- The time complexity of this algorithm is O(n3), where n is the number of vertices in the graph.

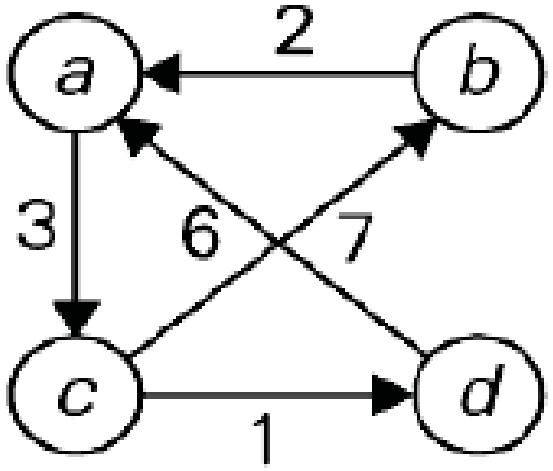
Floyd-Warshall algorithm

- Floyd's algorithm takes as input the cost matrix C(i,j).In a cost matrix

  C(i,j)=0 if i=j.
  C(i,j)=∞ if there is no edge between i and j.
  C(i,j)="cost of edge"

- Constructs solution through series of distance matrices $D^0, D^1, \ldots D^n$ using increasing subsets of the vertices allowed as intermediate.
- After n iterations it returns as output a distance matrix D[i,j] containing the cost of the shortest path from i to j.
- initially D[i,j] = C[i,j] On the k-th iteration, the algorithm determines shortest paths between every pair of vertices i,j that use only vertices among 1,…,k as intermediate

$$D^k[i,j] = \min \{D^{k-1}[i,j], D^{k-1}[i,k] + D^{k-1}[k,j]\}$$



$D^{(0)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

$D^{(1)} =$

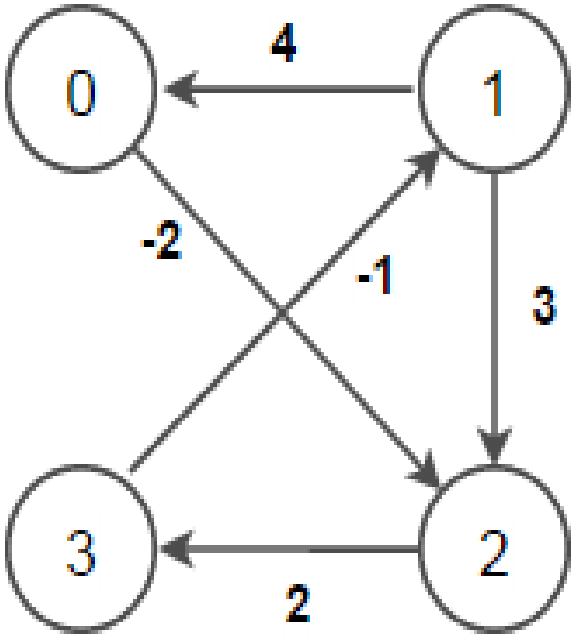
	a	b	c	d
a	0	∞	3	∞
b	2	0	**5**	∞
c	∞	7	0	1
d	6	∞	**9**	0

$$D^{(2)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{bmatrix} a & b & c & d \\ 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \mathbf{9} & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{bmatrix}$$

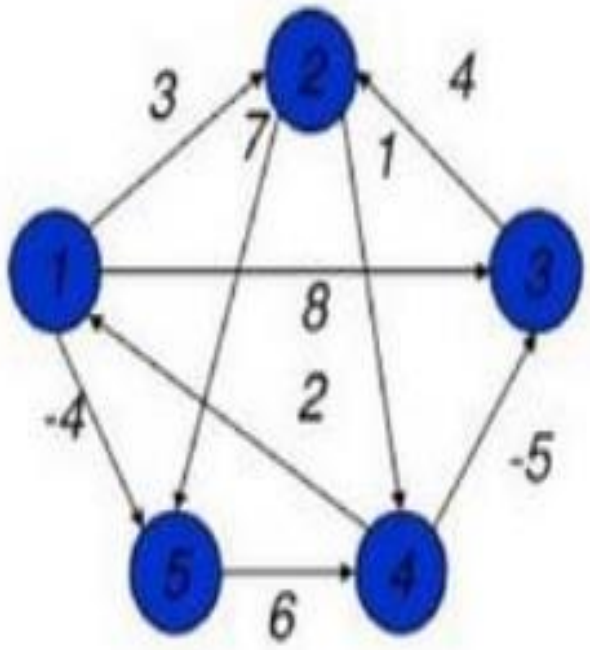
$$D^{(3)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{bmatrix} a & b & c & d \\ 0 & \mathbf{10} & 3 & \mathbf{4} \\ 2 & 0 & 5 & \mathbf{6} \\ 9 & 7 & 0 & 1 \\ 6 & \mathbf{16} & 9 & 0 \end{bmatrix}$$

$$D^{(4)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{bmatrix} a & b & c & d \\ 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ \mathbf{7} & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix}$$

Floyd-Warshall algorithm



All pairs shortest path problem



$$\begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

Algorithm Floyd-Warshall (n)

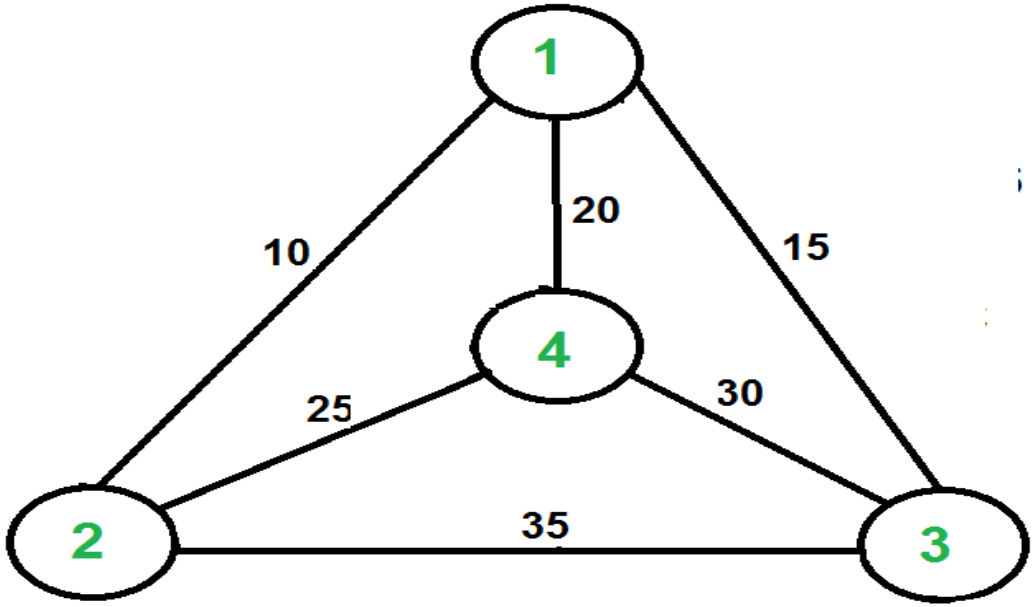
```
{
      int D[n][n];
      for (k = 1; k <=n; k++)
   {
       // Pick all vertices as source one by one
       for (i = 1; i < =n; i++)
       {
          // Pick all vertices as destination for the above picked source
```

```
        for (j = 1; j <=n; j++)
        {
            // If vertex k is on the shortest path from
            // i to j, then update the value of D[i][j]
            if (D[i][k] + D[k][j] < D[i][j])
               D[i][j] = D[i][k] + D[k][j];
        }
    }
} }
```

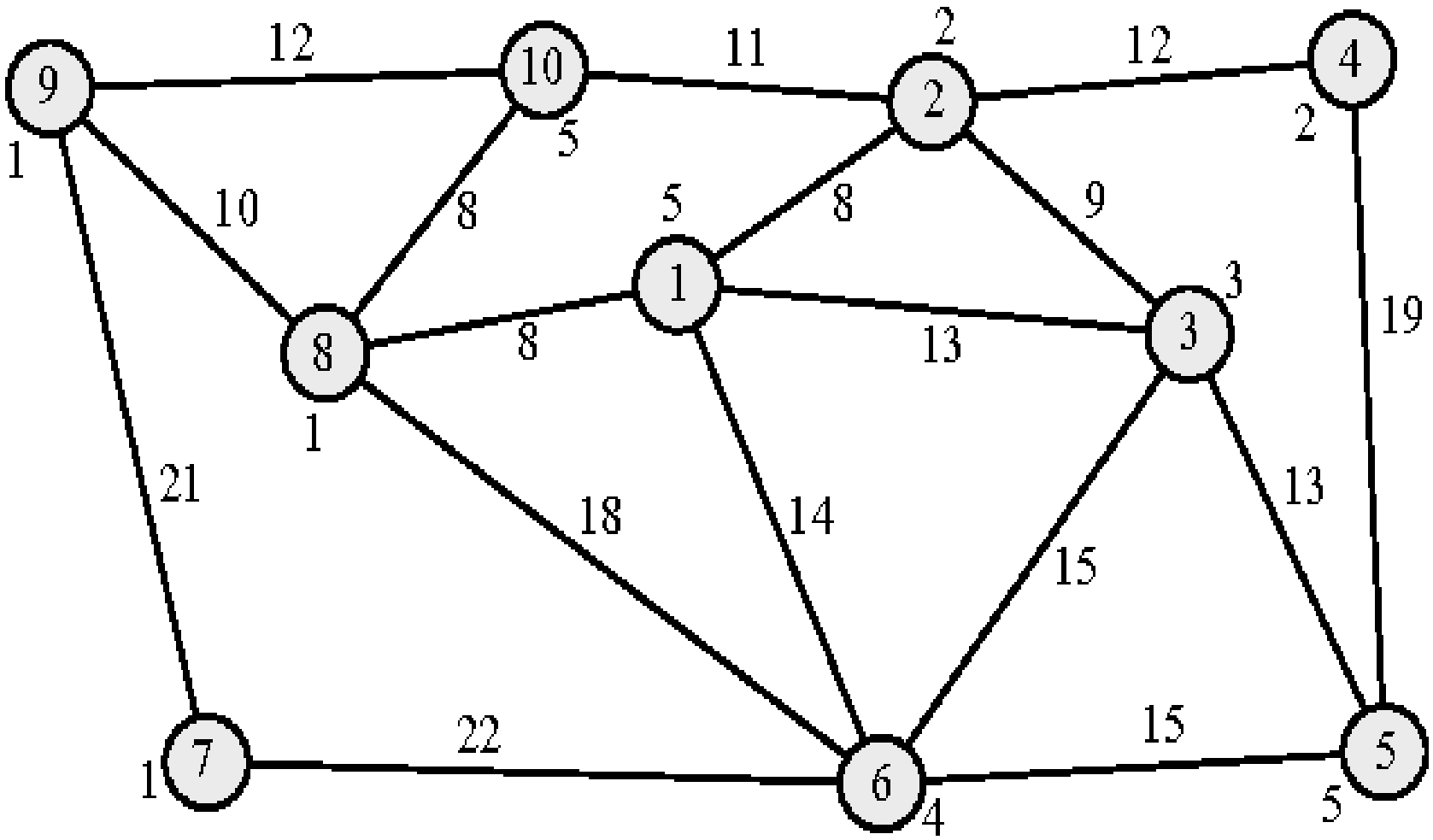
- **Time Complexity:** O(n3)

Travelling sales person problem

- Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible tour for a sales person, that visits every city exactly once and returns to the starting point.



A TSP tour in the graph is 1-2-4-3-1
cost of the tour is 10+25+30+15=80.


9
10
2
4
8
1
3
7
6
5
12
11
12
10
8
9
21
18
14
15
13
19
22
15
1
5
2
2
1
8
5
1
3
1
4
5

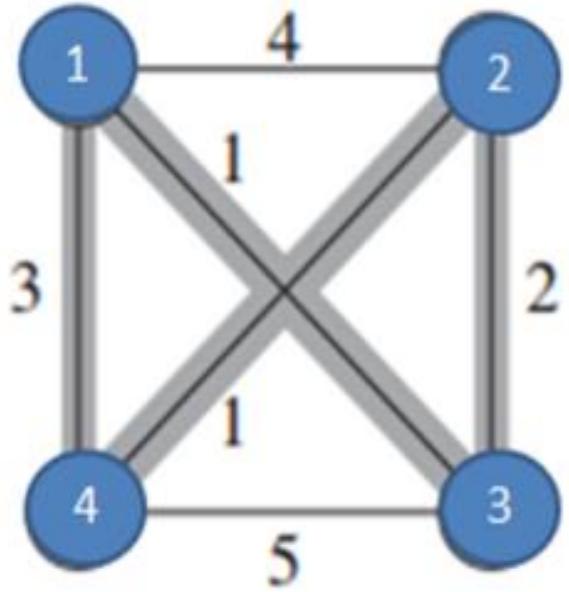
Travelling sales person problem

- Let us assume that vertex 1 is the starting node of the tour and cost(i,S) be the minimum cost path that starting from vertex 1 ,visiting all the vertices of set S exactly once and ending at vertex i.
- The function cost(i,S)is defined as follows:

$$cost(i, S) = \min_{K \in S} \{dist(i,K) + cost(K, S-\{K\})$$

- If S is null then $cost(i, \emptyset) = dist(i,1)$

Travelling sales person problem



	1	2	3	4
1	0	4	1	3
2	4	0	2	1
3	1	2	0	5
4	3	1	5	0

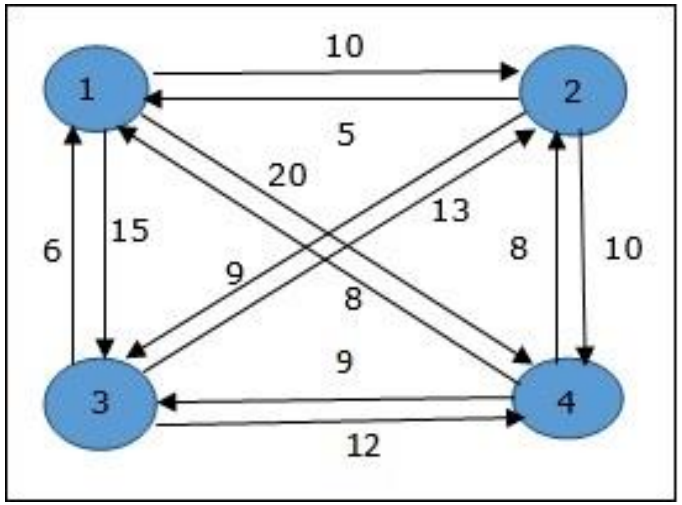
Travelling sales person problem

$$cost(i, S) = \min_{K \in S} \{dist(i,K) + cost(K, S-\{K\})\}$$

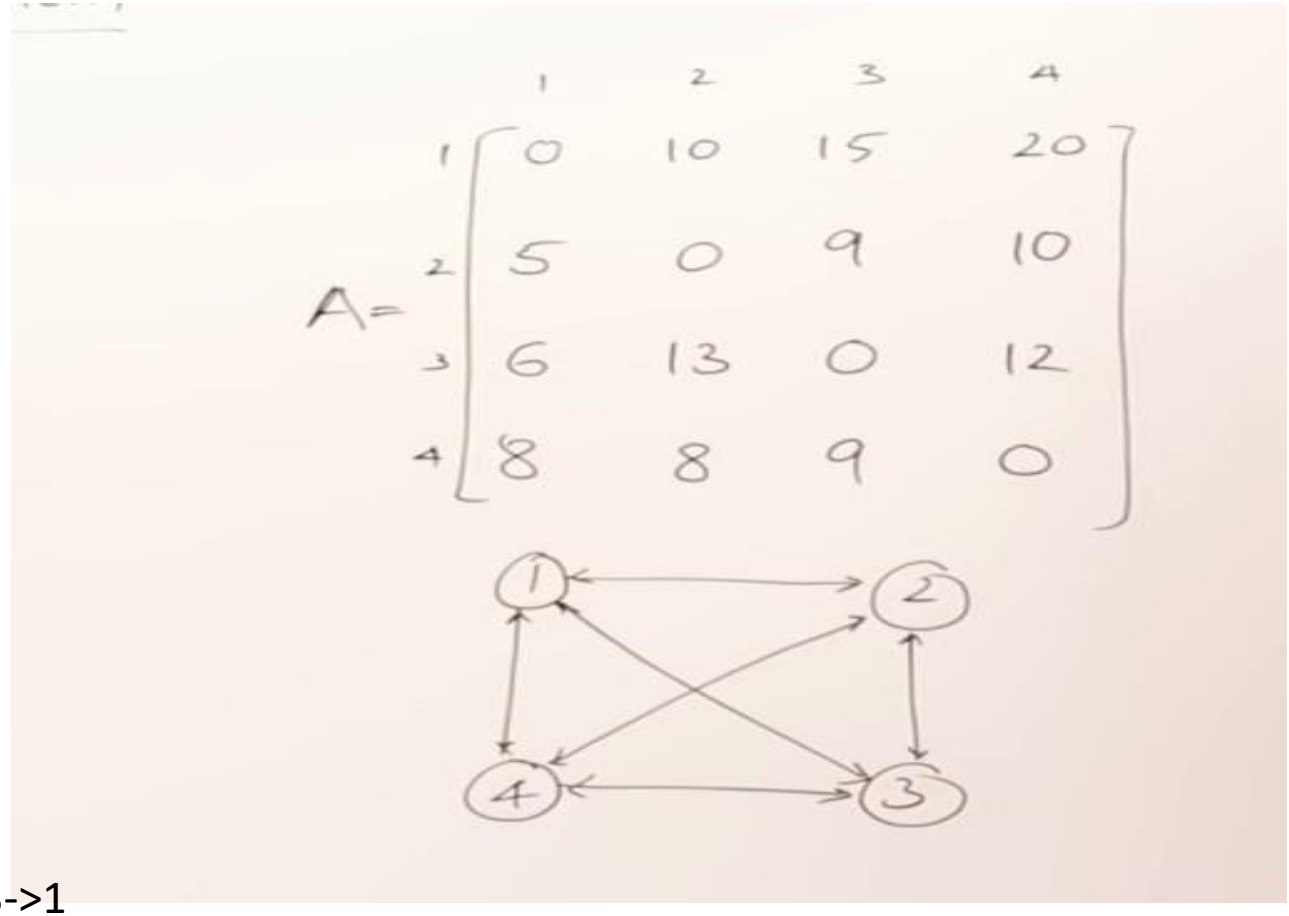
- $cost(2, \emptyset) = d(2,1) = 4$
- $cost(3, \emptyset) = d(3,1) = 1$
- $cost(4, \emptyset) = d(4,1) = 3$
- $cost(2, \{3\}) = dist(2, 3) + cost(3, \emptyset) = 2+1 = 3$
- $cost(2, \{4\}) = dist(2, 4) + cost(4, \emptyset) = 1+3 = 4$
- $cost(3, \{2\}) = dist(3, 2) + cost(2, \emptyset) = 2+4 = 6$
- $cost(3, \{4\}) = 8$
- $cost(4, \{2\}) = 5$
- $cost(4, \{3\}) = 6$

Travelling sales person problem

- cost(2, {3,4})=min{dist(2,3)+cost(3,4),dist(2,4)+cost(4,3)}=min{10,7}=7
- cost(3, {2,4})= min{dist(3,2)+cost(2,4),dist(3,4)+cost(4,2)}=6
- cost(4, {2,3})= min{dist(4,2)+cost(2,3),dist(4,3)+cost(3,2)}=4
- cost(1, {2,3,4})=min{dist(1,2)+cost(2,{3,4}),dist(1,3)+cost(3,{2,4},dist(1,4)+cost(4,{2,3})}=min{11,7,7}=7
- So the TSP tour is 1->3->2->4->1 or 1->4>2->3->1
- cost of the tour is 7

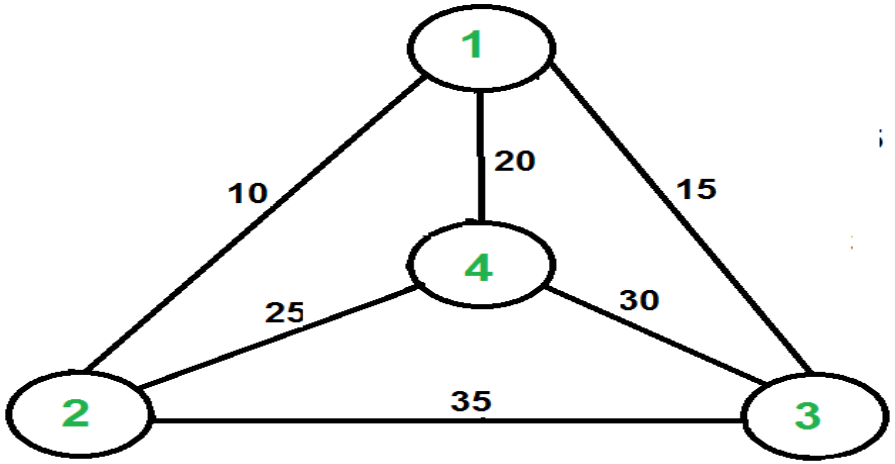


The minimum cost path is 1->2->4->3->1
Cost of the tour:35



The minimum cost path is 1->4->3->2->1
Cost of the tour:23

Travelling sales person problem



A TSP tour in the graph is 1-2-4-3-1
cost of the tour is 10+25+30+15 =80.